

BAŞLIK: GİT: GENEL TEKRAR SORULARI

TÜR: Çoktan Seçmeli Değerlendirme (Sorular 14.2 - 14.12 konularını kapsar)

SORULARIN CEVAPLARI EN ALTTA BULUNMAKTADIR.

1. Bir geliştirici yanlışlıkla force push yapıp başka bir ekip üyesinin commit'lerini görünmez hale getirdiğinde gerçek durum nedir?

- A) Veri kaybı asla telafi edilemez.
- B) Commit'ler kalıcı olarak silinmiştir, kurtarılamaz.
- C) Commit'ler silinmemiştir; sadece branch referansı değişmiştir ve reflog ile bulunabilir.
- D) Sadece Git yöneticisi kurtarabilir.

2. Paylaşılan bir branch üzerinde rebase yapıp push edilmesi neden krize yol açar?

- A) Rebase commit geçmişini yeniden yazdığı için diğer geliştiricilerin lokal branch'leri remote ile uyumsuz hale gelir.
- B) Rebase sadece dosya isimlerini değiştirir.
- C) Rebase branch'i otomatik siler.
- D) Rebase hiçbir zaman sorun çıkarmaz.

3. Production'a hatalı kod çıktığında önerilen yaklaşım nedir?

- A) Tüm branch'i silmek.
- B) Hiçbir şey yapmadan beklemek.
- C) reset --hard ile geçmişi silmek.
- D) Sorunlu commit'i tespit edip yeni bir revert commit oluşturarak sistemi stabilize etmek.

4. Yanlışlıkla silinen bir branch'in çalışmaları gerçekten kaybolmuş mudur?

- A) Sadece yedek alınmışsa kurtarılabilir.
- B) Hayır; branch silinse de commit'ler genellikle hâlâ mevcuttur ve reflog/log ile bulunup yeni bir branch'e geri getirilebilir.
- C) Evet, branch silinince commit'ler de kalıcı olarak silinir.
- D) Sadece GitHub üzerinden kurtarılabilir.

5. Genel Git kriz yönetim prensiplerinden biri nedir?

- A) Panik yapıp branch'i silmek.
- B) Hemen force push ile düzeltmeye çalışmak.
- C) Sorunu görmezden gelmek.
- D) Önce log ve reflog incelenir, history silinmeden çözüm aranır.

6. Git Flow stratejisindeki temel branch'ler (main, develop, feature, release, hotfix) hangi tür ekipler için tasarlanmıştır?

- A) Release odaklı, planlı geliştirme süreçlerine sahip kurumsal/büyük ekipler.

- B) Sadece tek kişilik projeler.
- C) Sadece açık kaynak projeleri.
- D) Hiçbir zaman kullanılmaz.

7. GitHub Flow ile Git Flow arasındaki temel fark nedir?

- A) GitHub Flow tek ana branch ve doğrudan pull request akışıyla daha sade/hızlı bir yapı sunar; Git Flow ise çok sayıda branch türü içerir.
- B) İkisi birebir aynıdır.
- C) GitHub Flow çok daha karmaşıktır.
- D) Git Flow sadece açık kaynak projelerde kullanılır.

8. Trunk Based Development yaklaşımının temel mantığı nedir?

- A) Sadece hafta sonları commit atmak.
- B) Uzun ömürlü feature branch'ler kullanmak.
- C) Hiç branch kullanmamak.
- D) Uzun ömürlü branch'ler olmadan, küçük ve sık değişikliklerle tek ana branch üzerinden çalışmak (feature flag'lerle desteklenir).

9. Kurumsal Git kullanımında 'Ana branch'e direkt commit atmak' neden en sık yapılan hatalardan biri olarak görülür?

- A) Çünkü hiçbir riski yoktur.
- B) Çünkü sadece disk alanını artırır.
- C) Çünkü hızı yavaşlatmaktan başka etkisi yoktur.
- D) Çünkü kod incelemeden (code review) geçmeden ana/production branch'e girip riski artırır.

10. Kurumsal Git ortamında Pull Request (PR) kültürünün temel amacı nedir?

- A) Sadece dosya boyutunu küçültmek.
- B) Kodun ana branch'e girmeden önce incelenmesini sağlayan bir kontrol noktası oluşturmak.
- C) Commit sayısını azaltmak için zorunlu bir engel koymak.
- D) Geliştiricilerin birbirini görmemesini sağlamak.

11. Ekip Git kuralları checklist'ine göre, ana branch'e giden her yol neyden geçmelidir?

- A) Hiçbir kontrolden geçmeden.
- B) Sadece e-posta onayından.
- C) Doğrudan commit'ten.
- D) Pull Request (PR) sürecinden.

12. Checklist'e göre 'Fix', 'Update', 'WIP' gibi ifadeler neden tek başına commit mesajı olarak kabul edilmez?

- A) Çünkü Git bu kelimeleri yasaklar.
- B) Çünkü İngilizce değildirler.
- C) Çünkü çok uzundurlar.

D) Çünkü neyin, neden değiştiğini açıklamazlar; gelecekteki geliştiriciler için anlamsız kalırlar.

13. Force push kuralları checklist'inde nasıl tanımlanmıştır?

- A) Sadece Cuma günleri yapılabilir.
 - B) Her zaman serbesttir.
 - C) Varsayılan olarak yasaktır; sadece kişisel branch'lerde ve gerekiyorsa ekip bilgilendirilerek yapılır.
 - D) Sadece senior geliştiriciler için tamamen serbesttir.
-

14. Checklist'e göre Junior geliştiricinin Git yetkileri hangileridir?

- A) Force push yapabilir.
 - B) Doğrudan production branch'e merge edebilir.
 - C) PR açar, review ister, ancak merge etmez.
 - D) Branch stratejisini o belirler.
-

15. CI kırırken (başarısız) checklist ne öngörür?

- A) Yeni merge yapılmaz, önce sistem stabilize edilir.
 - B) Yeni merge'lere devam edilir.
 - C) Ana branch silinir.
 - D) CI görmezden gelinir.
-

16. Git'te HEAD referansı ne anlama gelir?

- A) Bir dosyanın boyutunu.
 - B) Uzak (remote) sunucunun adresini.
 - C) Şu anda üzerinde çalışılan commit'i işaret eden referansı.
 - D) Bir commit mesajını.
-

17. Detached HEAD durumu ne zaman ortaya çıkar ve riski nedir?

- A) Sadece yeni bir repo oluşturulduğunda.
 - B) Sadece merge sırasında oluşur.
 - C) HEAD bir branch yerine doğrudan bir commit'e işaret ettiğinde; bu moddaki commit'ler branch oluşturulmazsa kaybolabilir.
 - D) Hiçbir zaman risk taşımaz.
-

18. Fast-forward merge ne zaman gerçekleşir?

- A) Hedef branch ilerideyken.
 - B) Hedef branch'in ileride olmadığı, yani branch pointer'ın doğrudan ileri taşınabildiği durumlarda; yeni bir merge commit oluşmaz.
 - C) Her merge işleminde otomatik olarak.
 - D) Sadece rebase sırasında.
-

19. Squash merge işlemi ne yapar?

- A) Bir branch'teki birden fazla commit'i tek bir commit olarak birleştirerek ana branch geçmişini sadeleştirir.
 - B) Bir branch'i tamamen siler.
 - C) Branch isimlerini değiştirir.
 - D) Commit mesajlarını şifreler.
-

20. Upstream branch kavramı ne işe yarar?

- A) Sadece conflict'leri otomatik çözer.
 - B) Yeni bir repo oluşturur.
 - C) Lokal branch'in takip ettiği remote branch'i tanımlar; tanımlıysa push/pull parametresiz çalışabilir.
 - D) Sadece görsel bir etikettir.
-

21. Merge işleminde commit geçmişine ne olur?

- A) Geçmiş tamamen silinir.
 - B) Tüm commit'ler yeniden yazılır.
 - C) Mevcut commit geçmişi korunur, gerekirse yeni bir merge commit oluşturulur.
 - D) Sadece son commit kalır.
-

22. Rebase işleminde commit'lere ne olur?

- A) Hiçbir şey değişmez.
 - B) Commit'ler yeni hash alır ve geçmiş yeniden yazılarak daha lineer bir history oluşur.
 - C) Commit'ler tamamen silinir.
 - D) Sadece commit mesajları değişir.
-

23. Neden paylaşılan (shared) bir branch'te rebase yapmak riskli kabul edilir?

- A) Rebase commit hash'lerini değiştirdiği için aynı branch'i kullanan diğer geliştiricilerin history'si bozulur.
 - B) Rebase sadece dosya isimlerini değiştirdiği için.
 - C) Rebase branch'i otomatik olarak siler.
 - D) Rebase hiçbir zaman risk taşımaz.
-

24. Genel kural olarak merge ve rebase hangi branch türlerinde tercih edilir?

- A) İkisi de her zaman aynı branch türünde kullanılır.
 - B) Merge kişisel branch'lerde, rebase paylaşılan branch'lerde.
 - C) Hiçbiri hiçbir zaman kullanılmamalıdır.
 - D) Merge paylaşılan/ortak branch'lerde (güvenlik için), rebase kişisel/henüz paylaşılmamış branch'lerde (temiz history için).
-

25. Soft reset hangi alanları etkiler?

- A) Sadece working directory'yi temizler.
- B) Hiçbir şeyi etkilemez.
- C) Hem commit'i hem staging area'yı hem working directory'yi siler.
- D) Sadece commit referansını geri alır; staging area ve working directory etkilenmez.

26. Mixed reset ile soft reset arasındaki fark nedir?

- A) Mixed reset ayrıca staging area'yı da temizler (working directory yine korunur), soft reset staging area'ya dokunmaz.
- B) Aralarında hiçbir fark yoktur.
- C) Mixed reset working directory'yi de siler.
- D) Soft reset daha risklidir.

27. Hard reset neden en riskli reset türüdür?

- A) Çünkü hiçbir şeyi değiştirmez.
- B) Çünkü commit, staging area ve working directory'yi birlikte geri alır; commit alınmamış tüm yerel değişiklikler kalıcı olarak kaybolur.
- C) Çünkü sadece commit mesajını değiştirir.
- D) Çünkü sadece uzak sunucuyu etkiler.

28. Reset ile revert arasındaki temel fark nedir?

- A) Reset her zaman güvenlidir.
- B) İkisi birebir aynı işi yapar.
- C) Reset commit'i silip history'yi değiştirir; revert ise geçmişini koruyarak hatalı commit'i tersine çeviren yeni bir commit oluşturur.
- D) Revert daha risklidir.

29. Production branch'lerde neden reset yerine revert tercih edilir?

- A) Revert daha az komut gerektirdiği için.
- B) Revert history'yi koruyup hangi değişikliğin geri alındığını net gösterirken, reset geçmişini silip izlenebilirliği bozar.
- C) Reset daha hızlı çalıştığı için.
- D) İkisi arasında hiçbir fark olmadığı için rastgele seçilir.

30. Git reflog neyi kaydeder?

- A) HEAD'in ve branch referanslarının geçmişte yaptığı tüm lokal hareketleri (checkout, commit, reset, rebase vb.).
- B) Sadece silinen dosyaları.
- C) Sadece uzak sunucudaki değişiklikleri.
- D) Sadece commit mesajlarının yazım hatalarını.

31. git log ile git reflog arasındaki fark nedir?

- A) git log reflog'dan daha fazla bilgi içerir.
 - B) git log sadece erişilebilir commit'leri gösterir; reflog artık branch'te görünmeyen (silinmiş/üzerine yazılmış) commit'leri de listeler.
 - C) reflog sadece uzak sunucuda çalışır.
 - D) İkisi birebir aynı bilgiyi gösterir.
-

32. Yanlışlıkla `reset --hard` kullanıldığında commit'ler gerçekten kaybolmuş mudur?

- A) Evet, kalıcı olarak silinirler.
 - B) Sadece yedekleme varsa kurtarılabilirler.
 - C) Hayır; commit'ler silinmemiştir, sadece branch referansı başka bir noktaya taşınmıştır ve reflog ile bulunabilir.
 - D) Sadece GitHub Destek ekibi kurtarabilir.
-

33. Reflog'un bir sınırlaması var mıdır?

- A) Hayır, sonsuza kadar her şeyi saklar.
 - B) Reflog sadece production branch'te çalışır.
 - C) Evet; reflog kayıtları sonsuza kadar tutulmaz, belirli bir süre sonra eski kayıtlar silinir.
 - D) Reflog remote sunucuda saklanır, kaybolmaz.
-

34. Production branch'lerde neden reset kullanılmaması önerilir?

- A) Çünkü reset commit geçmişini siler, bu da izlenebilirliği bozar ve rollback sürecini zorlaştırır.
 - B) Çünkü reset sadece Windows'ta çalışmaz.
 - C) Çünkü reset her zaman hatalıdır.
 - D) Çünkü reset çok yavaş çalışır.
-

35. Production'da bir hata tespit edildiğinde önerilen 'hotfix' yaklaşımı nasıl işler?

- A) Tüm proje silinip yeniden yazılır.
 - B) Production'dan bir hotfix branch açılır, düzeltme yapılır ve tamamlanınca tekrar production'a alınır.
 - C) Hiçbir işlem yapılmaz, hata kendiliğinden düzelir.
 - D) Doğrudan production'da rastgele değişiklik yapılır.
-

36. Production krizlerinde önerilen genel yaklaşım nedir?

- A) Hiçbir aksiyon almamak.
 - B) Production branch'i silmek.
 - C) Panik yapmadan sorunlu commit'i tespit edip revert ile sistemi stabilize etmek, kök neden analizini sonra yapmak.
 - D) Panik yapıp hemen reset atmak.
-

37. İyi bir branch isminin sahip olması gereken özellik nedir?

- A) Kısa, anlamlı ve işin amacını net ifade eden bir isim (örn. feature/user-login-validation).
 - B) Her zaman 'test' veya 'deneme' olması.
 - C) Rastgele karakterlerden oluşan bir isim (örn. abc123).
 - D) Sadece büyük harflerle yazılmış bir isim.
-

38. `hotfix/` branch kategorisi hangi amaçla kullanılır?

- A) Bakım işleri için.
- B) Deneme çalışmaları için.
- C) Yeni özellik geliştirme için.

D) Production'daki acil müdahaleler için.

39. İyi bir commit mesajının cevaplaması gereken sorular nelerdir?

- A) Sadece commit'i kimin attığı.
 - B) Ne değişti, neden değişti ve varsa etkisi ne.
 - C) Sadece 'ne değişti'.
 - D) Sadece tarih ve saat.
-

40. 'fix', 'update', 'wip' gibi commit mesajları neden kötü örnek olarak gösterilir?

- A) Çünkü Git tarafından reddedilirler.
 - B) Çünkü değişikliğin ne olduğu veya neden yapıldığı hakkında hiçbir bilgi vermezler.
 - C) Çünkü çok uzundurlar.
 - D) Çünkü İngilizce yazılmışlardır.
-

41. Continuous Integration (CI) sürecinin temel amacı nedir?

- A) Commit sayısını azaltmak.
 - B) Kodun elle test edilmesini zorunlu kılmak.
 - C) Sadece dokümantasyon üretmek.
 - D) Koda yapılan her değişikliğin otomatik olarak test edilerek hataların erken aşamada yakalanmasını sağlamak.
-

42. Bir Pull Request açıldığında CI/CD süreciyle ilişkisi nedir?

- A) PR açıldığında otomatik testler tetiklenir; testler başarısızsa PR merge edilmeyerek ana branch korunur.
 - B) CI sadece production branch'te çalışır, PR'larda çalışmaz.
 - C) Hiçbir ilişkisi yoktur.
 - D) PR otomatik olarak silinir.
-

43. Ana branch'in 'her zaman deploy edilebilir durumda olması' ilkesi neden önemlidir?

- A) Çünkü branch'i küçük tutar.
 - B) Çünkü ana branch'e yapılan her merge potansiyel bir deploy anlamına gelir; bu yüzden girişler sıkı kontrol edilmelidir.
 - C) Çünkü commit sayısını azaltır.
 - D) Çünkü estetik bir tercihtir.
-

44. CI başarısızken (fail olan build varken) neden merge yapılmaması önerilir?

- A) Çünkü merge işlemi teknik olarak imkansız hale gelir.
 - B) Çünkü hiçbir zaman önemli değildir.
 - C) Çünkü CI süreci otomatik olarak devre dışı kalır.
 - D) Çünkü hatalı kod ana branch'e/production'a girme riski taşır ve rollback ihtiyacı doğurabilir.
-

45. Checklist'e göre Junior geliştiricinin Git yetkileri arasında olmayan nedir?

- A) Ana branch'e merge yetkisi.
- B) Feature/bugfix branch açmak.
- C) Code review istemek.
- D) Pull request açmak.

46. Mid seviye geliştiricinin sorumlulukları arasında hangisi yer alır?

- A) Production süreçlerine tek başına karar vermek.
- B) Sadece kod yazmak, hiçbir inceleme yapmamak.
- C) Branch stratejisini tek başına belirlemek.
- D) Başkalarının PR'larını review etmek ve merge conflict çözebilmek.

47. Senior Developer / Tech Lead rolünün sorumlulukları arasında hangisi yer alır?

- A) Sadece hata bildirmek.
- B) Sadece PR açmak.
- C) Branch stratejisini belirlemek, Git workflow'larını tanımlamak ve Git krizlerinde yönlendirici olmak.
- D) Hiçbir yetkisi yoktur.

48. Rol bazlı Git sorumluluklarının netleştirilmesinin kurumsal faydası nedir?

- A) Hiçbir faydası yoktur.
- B) Yetki karmaşasını azaltır, yanlışlıkla production'ın kırılmasını önler ve ekip içi güveni artırır.
- C) Sadece bürokrasiyi artırır.
- D) Geliştirme hızını yavaşlatmaktan başka etkisi yoktur.

CEVAP ANAHTARI

1-C 2-A 3-D 4-B 5-D 6-A 7-A 8-D 9-D 10-B 11-D 12-D 13-C 14-C 15-A 16-C 17-C 18-B 19-A 20-C
21-C 22-B 23-A 24-D 25-D 26-A 27-B 28-C 29-B 30-A 31-B 32-C 33-C 34-A 35-B 36-C 37-A 38-D
39-B 40-B 41-D 42-A 43-B 44-D 45-A 46-D 47-C 48-B